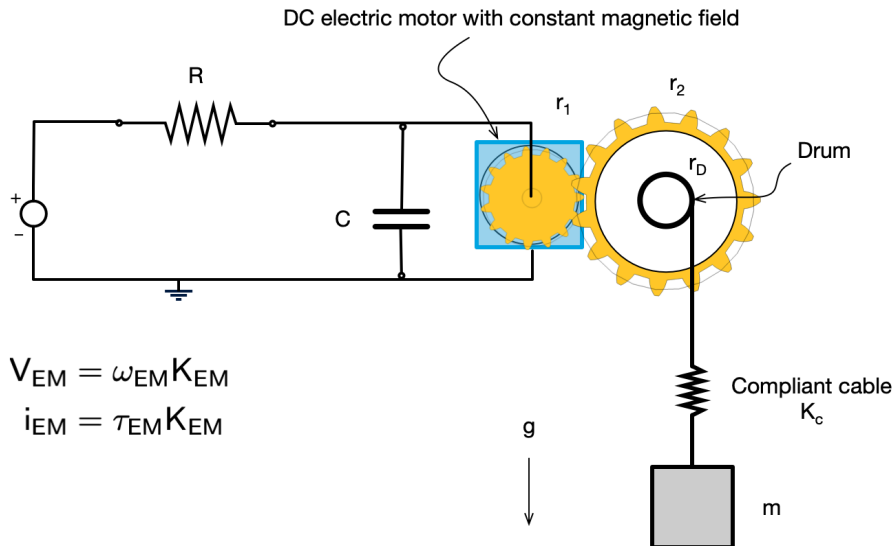


Exercise: Multidomain electro-mechanical system

The cage of an elevator is hoisted by a long cable wound over a drum driven through a gear-set by an electric motor. The motor is relay-operated (i.e., either on or off) and the resulting abrupt transients cause the cage to oscillate on the hoisting cable. Because the cable has low internal friction, the oscillations persist for many cycles.

Please note that the motor armature resistance and inductance may be neglected and also drum and gear inertia may be neglected.

To solve this problem it has been proposed to introduce an electrical R-C filter between the relay and the motor terminals. The designer claims that this will smooth the transient, thereby reducing the oscillation amplitude to acceptable levels. Your task is to evaluate this proposal. Does electrical filtering help?



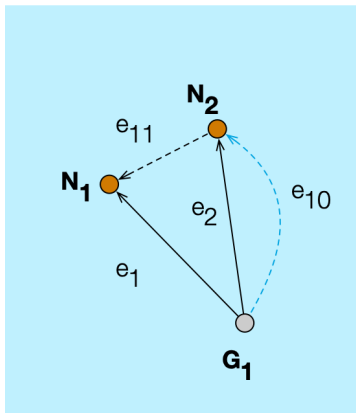
- The mass is an A-type element (1 edge)
- The cable is a T-type element (spring) (1 edge)
- The drum set is transformer (2 edges)
- The gear set is transformer (2 edges)
- The electric motor is a coupler/gyrator (2 edges)
- The capacitance is a A-type element (1 edge)
- The resistor is a D-type element (1 edge)
- The VOLTage source is an A-type source (1 edge)

In total we have 11 edges thus 11 constitutive equations

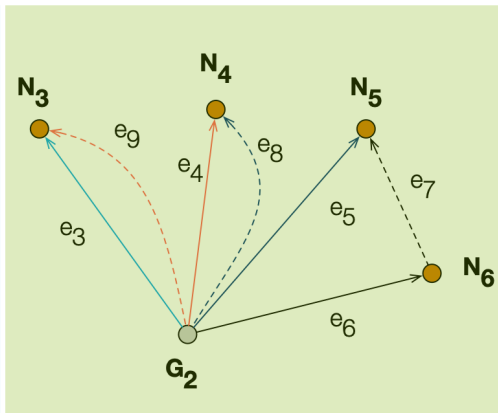
Linear graph and normal tree

System Graph with the normal tree is shown in the picture

ELECTRICAL DOMAIN



MECHANICAL DOMAIN



Definitions

Definition of across and through variables

```
clc
clear
close all
%
% library folder to use the utility functions
addpath('MBlibs')
```

Definition of problem variables

```
syms t

N = 11; % Number of edges

% alpha_vars = sym("alpha_%d", [N,0]);
% tau_vars = sym("tau_%d", [N,0]);
alpha_vars = sym.empty(1,0);
tau_vars = sym.empty(1,0);
% number of functions
for k = 1:N
    eval(sprintf('syms tau%d(t)', k));
    eval(sprintf('syms alpha%d(t)', k));
    % Optionally initialize a value: eval(sprintf('alpha%d(t) = 0;', k));
    % Build a row vector of the symbolic functions evaluated at t
    alpha_vars(k) = str2sym(sprintf('alpha%d(t)', k));
    tau_vars(k) = str2sym(sprintf('tau%d(t)', k));
end

tau_vars = tau_vars.';
alpha_vars = alpha_vars.';
```

Constitutive equations

We define the constitutive equations for each element (i.e. edge)

NOTE: in the constitutive equation of the mass we added the contribution of the gravity as external force acting on the mass without adding the corresponding element in the tree.

```
% we add an extra variable which is the spring displacement and one
% additional constitutive equations

syms K_s m V0 r1 r2 r C R K_emf gear_ratio g

% gear_ratio = r1/r2;

eqns_const = [ % Voltage source
               alpha1(t) == V0
               % capacitor
               diff(alpha2(t),t) == 1/C*tau2(t)
               % resistor
               alpha11(t) == R*tau11(t)
               % electric motor
               alpha3(t) == 1/K_emf*alpha10(t)
               tau3(t) == -K_emf*tau10(t)
               % gear train
               alpha4(t) == -gear_ratio*alpha9(t)
               tau4(t) == 1/gear_ratio*tau9(t)
               % drum
               alpha5(t) == r*alpha8(t)
               tau5(t) == -1/r*tau8(t)
               % spring: defined as differential being alpha8 a velocity
               diff(tau7(t),t) == K_s*alpha8(t)
               % mass
               % NOTE: we also added the gravity as external force
               diff(alpha6(t),t) == 1/m*tau6(t)-g
             ]
```

```
eqns_const =
```

$$\left(\begin{array}{l} \alpha_1(t) = V_0 \\ \frac{\partial}{\partial t} \alpha_2(t) = \frac{\tau_2(t)}{C} \\ \alpha_{11}(t) = R \tau_{11}(t) \\ \alpha_3(t) = \frac{\alpha_{10}(t)}{K_{\text{emf}}} \\ \tau_3(t) = -K_{\text{emf}} \tau_{10}(t) \\ \alpha_4(t) = -\text{gear}_{\text{ratio}} \alpha_9(t) \\ \tau_4(t) = \frac{\tau_9(t)}{\text{gear}_{\text{ratio}}} \\ \alpha_5(t) = r \alpha_8(t) \\ \tau_5(t) = -\frac{\tau_8(t)}{r} \\ \frac{\partial}{\partial t} \tau_7(t) = K_s \alpha_8(t) \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_6(t)}{m} - g \end{array} \right)$$

Loop and node equations

Incidence matrix

The incidence matrix has two singular rows in these cases. We write directly the matrix without singular rows.

```
%      e1 e2 e3 e4 e5 e6 e7 e8 e9 e10 e11
IM = [ 1   0  0  0  0  0  0  0  0  0  0  1 ;    % N1
      0   1  0  0  0  0  0  0  0  1 -1  0 ;    % N2
      0   0  1  0  0  0  0  0  1  0  0  0 ;    % N3
      0   0  0  1  0  0  0  1  0  0  0  0 ;    % N4
      0   0  0  0  1  0  1  0  0  0  0  0 ;    % N5
      0   0  0  0  0  1 -1  0  0  0  0  0 ;    % N6
     -1  -1  0  0  0  0  0  0  0  0  0  0 ;    % G1
      0   0 -1 -1 -1 -1  0 -1 -1 -1  0  0 ;    % G2
    ]
```

```
IM = 8x11
    1     0     0     0     0     0     0     0     0     0     0     1
    0     1     0     0     0     0     0     0     0     1    -1
    0     0     1     0     0     0     0     0     1     0     0
    0     0     0     1     0     0     0     1     0     0     0
    0     0     0     0     1     0     1     0     0     0     0
    0     0     0     0     0     1    -1     0     0     0     0
   -1    -1     0     0     0     0     0     0     0     0     0
    0     0    -1    -1    -1    -1     0    -1    -1    -1     0
```

```
[nn,ne] = size(IM);
```

```
% Check system consistency
if ~(length(alpha_vars) == ne )
    error 'Inconsistent dimensions'
```

```
end
% row reduced Echelon Form.
rref(IM)
```

```
ans = 8x11
    1     0     0     0     0     0     0     0     0     0     1
    0     1     0     0     0     0     0     0     0     0    -1
    0     0     1     0     0     0     0     0     1     0     0
    0     0     0     1     0     0     0     1     0     0     0
    0     0     0     0     1     0     1     0     0     0     0
    0     0     0     0     0     1    -1     0     0     0     0
    0     0     0     0     0     0     0     0     0     1     0
    0     0     0     0     0     0     0     0     0     0     0
```

Two rows are linearly dependent since they correspond to the ground

```
rank(IM)
```

```
ans =
7
```

```
%IM*tau_vars
% Number of elements, number of branches and number of chords
% Reduced Incidence Matrix
nb = nn-2; % n_nodes-1
ne ;      % number of edges
nc = ne-nb; % number of chords
%reduced IM: ground node is deleted
```

```
A = sym(IM(1:nb,1:ne))
```

```
A =
(1 0 0 0 0 0 0 0 0 0 1)
(0 1 0 0 0 0 0 0 0 1 -1)
(0 0 1 0 0 0 0 0 1 0 0)
(0 0 0 1 0 0 0 1 0 0 0)
(0 0 0 0 1 0 1 0 0 0 0)
(0 0 0 0 0 1 -1 0 0 0 0)
```

```
% compute Af matrix: Af = [Ib | Ac]
Af = rref(A)
```

```
Af =
(1 0 0 0 0 0 0 0 0 0 1)
(0 1 0 0 0 0 0 0 0 1 -1)
(0 0 1 0 0 0 0 0 1 0 0)
(0 0 0 1 0 0 0 1 0 0 0)
(0 0 0 0 1 0 1 0 0 0 0)
(0 0 0 0 0 1 -1 0 0 0 0)
```

```
Ac = Af(:,nb+1:end) % Ac = Af(1:nb,nb+1:ne);
```

$A_c =$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Node equations (i.e. Newton Equations at nodes)

Node equations are derived from the rows of rowEchelon form of the reduced incidence matrix: they are the **fundamental cutsets**

$$\mathbf{A}_f = [\mathbf{I}_b | \mathbf{A}_C].$$

```
node_eqns = tau_vars(1:nb)==-Ac*tau_vars(nb+1:ne)
```

node_eqns =

$$\begin{pmatrix} \tau_1(t) = -\tau_{11}(t) \\ \tau_2(t) = \tau_{11}(t) - \tau_{10}(t) \\ \tau_3(t) = -\tau_9(t) \\ \tau_4(t) = -\tau_8(t) \\ \tau_5(t) = -\tau_7(t) \\ \tau_6(t) = \tau_7(t) \end{pmatrix}$$

Circuits equations/Loop equations

We find the **fundamental circuits** from the A_f matrix. The B_f matrix which is the circuit loop matrix is derived as null space of matrix A_f .

$$\mathbf{B}_f = [\mathbf{B}_b | \mathbf{I}_c]$$

```
Bf = null(Af).'
```

$B_f =$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & -1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

```
Af*Bf'
```

ans =

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

We compute the Bb matrix exploiting the principle of

$$Bb = -Ac'$$

$$Bb =$$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & -1 & 1 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$$Ic = Bf(:, nc+1:ne)$$

$$Ic =$$

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Loop equations

```
% Bb = Bf(:,1:nb)
% Ic = Bf(:,nc:ne)
```

Branch across variables as function of across chords variables

$$\text{loop_eqns} = \alpha_vars(nb+1:ne) == -Bb * \alpha_vars(1:nb)$$

$$\text{loop_eqns} =$$

$$\begin{pmatrix} \alpha_7(t) = \alpha_5(t) - \alpha_6(t) \\ \alpha_8(t) = \alpha_4(t) \\ \alpha_9(t) = \alpha_3(t) \\ \alpha_{10}(t) = \alpha_2(t) \\ \alpha_{11}(t) = \alpha_1(t) - \alpha_2(t) \end{pmatrix}$$

DAE system of equations

Dependent and independent coordinates

The **dependent variables** are the set of **chord across coordinates** and of the **through branch coordinates**:

```
% dependent set of equations from loop and node equations
qvars_d = [lhs(node_eqns); lhs(loop_eqns)]
```

$$qvars_d =$$

$$\begin{pmatrix} \tau_1(t) \\ \tau_2(t) \\ \tau_3(t) \\ \tau_4(t) \\ \tau_5(t) \\ \tau_6(t) \\ \alpha_7(t) \\ \alpha_8(t) \\ \alpha_9(t) \\ \alpha_{10}(t) \\ \alpha_{11}(t) \end{pmatrix}$$

```
% independent set of DAE variables
allvars = [alpha_vars;tau_vars;];
dae_vars = setdiff(allvars,qvars_d) % dae_vars
```

```
dae_vars =
```

$$\begin{pmatrix} \alpha_1(t) \\ \alpha_2(t) \\ \alpha_3(t) \\ \alpha_4(t) \\ \alpha_5(t) \\ \alpha_6(t) \\ \tau_7(t) \\ \tau_8(t) \\ \tau_9(t) \\ \tau_{10}(t) \\ \tau_{11}(t) \end{pmatrix}$$

Assembling and solving equations

```
% Substitute loop equations into constitutive equations
%subs( eqns_const, sym2cell(lhs(eqns_loop)),sym2cell(rhs(eqns_loop)))
tmp = subs( eqns_const, lhs(loop_eqns),rhs(loop_eqns));
% substitute node equations
dae_eqns = subs( tmp, lhs(node_eqns), rhs(node_eqns))
```

```
dae_eqns =
```


$$\left(\begin{array}{l} \alpha_1(t) = V_0 \\ \frac{\partial}{\partial t} \alpha_2(t) = -\frac{\tau_{10}(t) - \tau_{11}(t)}{C} \\ \alpha_1(t) - \alpha_2(t) = R \tau_{11}(t) \\ \alpha_3(t) = \frac{\alpha_2(t)}{K_{\text{emf}}} \\ -\tau_9(t) = -K_{\text{emf}} \tau_{10}(t) \\ \alpha_4(t) = -\text{gear}_{\text{ratio}} \alpha_3(t) \\ -\tau_8(t) = \frac{\tau_9(t)}{\text{gear}_{\text{ratio}}} \\ \alpha_5(t) = r \alpha_4(t) \\ -\tau_7(t) = -\frac{\tau_8(t)}{r} \\ \frac{\partial}{\partial t} \tau_7(t) = K_s \alpha_4(t) \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{m} - g \end{array} \right)$$

Check dimensions

```
if ~(length(qvars_d)==N)
    fprintf('dep. vars %d <> n. elements %d',length(qvars_d),N)
    error 'Inconsistent number of dependent variables'
end
if ~(length(eqns_const)==N)
    fprintf('n. constitutive eqns. %d <> n. elements %d',length(eqns_const),N);
    error 'Constitutive equations must be equal to number of dependent variables'
end
if ~(length(dae_eqns)==length(dae_vars))
    fprintf('n.dae eqns %d <> n. eqns vars %d',length(dae_eqns),length(dae_vars))
    error 'Inconsistent set of equations'
end
```

Equations manipulation: index reduction

```
[E_mat, G_vec] = massMatrixForm(dae_eqns,dae_vars)
```

```
E_mat =
```

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

G_vec =

$$\begin{pmatrix} V_0 - \alpha_1(t) \\ -\frac{\tau_{10}(t) - \tau_{11}(t)}{C} \\ \alpha_2(t) - \alpha_1(t) + R \tau_{11}(t) \\ \frac{\alpha_2(t)}{K_{\text{emf}}} - \alpha_3(t) \\ \tau_9(t) - K_{\text{emf}} \tau_{10}(t) \\ -\alpha_4(t) - \text{gear}_{\text{ratio}} \alpha_3(t) \\ \tau_8(t) + \frac{\tau_9(t)}{\text{gear}_{\text{ratio}}} \\ r \alpha_4(t) - \alpha_5(t) \\ \tau_7(t) - \frac{\tau_8(t)}{r} \\ K_s \alpha_4(t) \\ \frac{\tau_7(t)}{m} - g \end{pmatrix}$$

Equations/Variables sorting

Use Dulmage-Mendelsohn algorithm to reorder the equations

```
[p,q,~,~,~,~] = dmperm(double(E_mat));
% Rebuilt sorted system
E_sorted = E_mat(p, :) % ONLY equations order is changed
```

E_sorted =

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

```
G_sorted = G_vec(p) ;
% not full rank: system is dae
n_ode = rank(E_sorted)
```

```
n_ode =
3
```

```
dae_vars
```

```
dae_vars =
```

$$\begin{pmatrix} \alpha_1(t) \\ \alpha_2(t) \\ \alpha_3(t) \\ \alpha_4(t) \\ \alpha_5(t) \\ \alpha_6(t) \\ \tau_7(t) \\ \tau_8(t) \\ \tau_9(t) \\ \tau_{10}(t) \\ \tau_{11}(t) \end{pmatrix}$$

```
%dae_vars_sorted = flip(dae_vars(q))
% Equations are reordered
eqns_sorted = E_sorted*diff(dae_vars,t)==G_sorted
```

```
eqns_sorted =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} \alpha_2(t) = -\frac{\tau_{10}(t) - \tau_{11}(t)}{C} \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{m} - g \\ \frac{\partial}{\partial t} \tau_7(t) = K_s \alpha_4(t) \\ 0 = V_0 - \alpha_1(t) \\ 0 = \alpha_2(t) - \alpha_1(t) + R \tau_{11}(t) \\ 0 = \frac{\alpha_2(t)}{K_{\text{emf}}} - \alpha_3(t) \\ 0 = \tau_9(t) - K_{\text{emf}} \tau_{10}(t) \\ 0 = -\alpha_4(t) - \text{gear}_{\text{ratio}} \alpha_3(t) \\ 0 = \tau_8(t) + \frac{\tau_9(t)}{\text{gear}_{\text{ratio}}} \\ 0 = r \alpha_4(t) - \alpha_5(t) \\ 0 = \tau_7(t) - \frac{\tau_8(t)}{r} \end{pmatrix}$$

We get the ode equations which are the first n_ode equations

```
% Extract odes equations
ode_eqns = eqns_sorted(1:n_ode)
```

$$\begin{pmatrix} \frac{\partial}{\partial t} \alpha_2(t) = -\frac{\tau_{10}(t) - \tau_{11}(t)}{C} \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{m} - g \\ \frac{\partial}{\partial t} \tau_7(t) = K_s \alpha_4(t) \end{pmatrix}$$

Extract the differential variables from the ode equations

```
% Extract the vector of differential variables
ode_vars = get_ode_vars(dae_vars,ode_eqns)
```

$$\begin{pmatrix} \alpha_2(t) \\ \alpha_6(t) \\ \tau_7(t) \end{pmatrix}$$

Now the remaining are the algebraic variables

```
% Extract the vector of algebraic variables
alg_vars = setdiff(dae_vars,ode_vars)
```

alg_vars =

$$\begin{pmatrix} \alpha_1(t) \\ \alpha_3(t) \\ \alpha_4(t) \\ \alpha_5(t) \\ \tau_8(t) \\ \tau_9(t) \\ \tau_{10}(t) \\ \tau_{11}(t) \end{pmatrix}$$

And we get the algebraic equations

```
% Extract algebraic equations
alg_eqns = eqns_sorted(n_ode+1:end)
```

alg_eqns =

$$\begin{pmatrix} 0 = V_0 - \alpha_1(t) \\ 0 = \alpha_2(t) - \alpha_1(t) + R \tau_{11}(t) \\ 0 = \frac{\alpha_2(t)}{K_{\text{emf}}} - \alpha_3(t) \\ 0 = \tau_9(t) - K_{\text{emf}} \tau_{10}(t) \\ 0 = -\alpha_4(t) - \text{gear}_{\text{ratio}} \alpha_3(t) \\ 0 = \tau_8(t) + \frac{\tau_9(t)}{\text{gear}_{\text{ratio}}} \\ 0 = r \alpha_4(t) - \alpha_5(t) \\ 0 = \tau_7(t) - \frac{\tau_8(t)}{r} \end{pmatrix}$$

```
% % First differentiation:
% d_alg_eqns1 = diff(alg_eqns,t);
% %
% % assemble system
% dae_eqns1 = [ode_eqns;d_alg_eqns1]
% % Compute the E matrix and G vector for new system
% [E_mat1,G_vec1] = massMatrixForm(dae_eqns1,dae_vars);
%
% % Compute LU
% [L1,U1,P1] = lu(E_mat1);
% L1;
% U1
% Check the rank of the system: is full
% the system is index 1
%n_ode1 = rank(U1)
%length(dae_vars)==n_ode1
% The system reduced to pure ode
% Since we made one differentiation the system is index dae 1
```

```
%eqns_ode_reduced = diff(dae_vars,t)==U1\L1\ (P1*G_vec1)
% The list of invariants is
%invariants = [alg_eqns];
```

Numerical solution

Since the system is a index-1 DAE system we can try to solve the invariant equations for the algebraic variables.

```
ode_eqns_aug = [ode_eqns]
```

```
ode_eqns_aug =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} \alpha_2(t) = -\frac{\tau_{10}(t) - \tau_{11}(t)}{C} \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{m} - g \\ \frac{\partial}{\partial t} \tau_7(t) = K_s \alpha_4(t) \end{pmatrix}$$

Algebraic equations are linear in the algebraic variables. The linear system can be solved and substituted in the ode

```
loop_eqns
```

```
loop_eqns =
```

$$\begin{pmatrix} \alpha_7(t) = \alpha_5(t) - \alpha_6(t) \\ \alpha_8(t) = \alpha_4(t) \\ \alpha_9(t) = \alpha_3(t) \\ \alpha_{10}(t) = \alpha_2(t) \\ \alpha_{11}(t) = \alpha_1(t) - \alpha_2(t) \end{pmatrix}$$

```
node_eqns
```

```
node_eqns =
```

$$\begin{pmatrix} \tau_1(t) = -\tau_{11}(t) \\ \tau_2(t) = \tau_{11}(t) - \tau_{10}(t) \\ \tau_3(t) = -\tau_9(t) \\ \tau_4(t) = -\tau_8(t) \\ \tau_5(t) = -\tau_7(t) \\ \tau_6(t) = \tau_7(t) \end{pmatrix}$$

```
if ~(length(alg_eqns) == length(alg_vars))
    warning 'Inconsistent system'
end

%[AA, bb] = equationsToMatrix(invariants, alg_vars);
%alg_vars==AA\bb
```

Solve the algebraic equation

```
syms y [length(alg_vars),1]
tmp = subs( alg_eqns, alg_vars, y);
sol_alg = solve(tmp,y)
```

```
sol_alg = struct with fields:
  y1: V0
  y2: alpha2(t)/K_emf
  y3: -(gear_ratio*alpha2(t))/K_emf
  y4: -(gear_ratio*r*alpha2(t))/K_emf
  y5: r*tau7(t)
  y6: -gear_ratio*r*tau7(t)
  y7: -(gear_ratio*r*tau7(t))/K_emf
  y8: (V0 - alpha2(t))/R
```

alg_vars

```
alg_vars =
```

$$\begin{pmatrix} \alpha_1(t) \\ \alpha_3(t) \\ \alpha_4(t) \\ \alpha_5(t) \\ \tau_8(t) \\ \tau_9(t) \\ \tau_{10}(t) \\ \tau_{11}(t) \end{pmatrix}$$

This is the final system of ordinary differential equations to be solved

```
reduced_ode = subs( subs(ode_eqns , alg_vars, y),sol_alg)
```

```
reduced_ode =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} \alpha_2(t) = \frac{\frac{V_0 - \alpha_2(t)}{R} + \frac{\text{gear_ratio } r \tau_7(t)}{K_{\text{emf}}}}{C} \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{m} - g \\ \frac{\partial}{\partial t} \tau_7(t) = -\frac{K_s \text{gear_ratio } \alpha_2(t)}{K_{\text{emf}}} \end{pmatrix}$$

Numerical data

```
% system data
data.R = 10; % Ohm
data.C = 0.1; % farads
data.m = 200; % kg
data.K_s = 200000; % N/m
data.r = 0.05; % (m)
data.gear_ratio = 0.02; % ( )
data.K_emf = 0.03; % (N/ampere)
data.g = 9.81; % m/s^2
%data.V0 = 24;
```

```
% substitute data
ode_red = subs( reduced_ode,data)
```

```
ode_red =
```

$$\begin{pmatrix} \frac{\partial}{\partial t} \alpha_2(t) = V_0 - \alpha_2(t) + \frac{\tau_7(t)}{3} \\ \frac{\partial}{\partial t} \alpha_6(t) = \frac{\tau_7(t)}{200} - \frac{981}{100} \\ \frac{\partial}{\partial t} \tau_7(t) = -\frac{400000 \alpha_2(t)}{3} \end{pmatrix}$$

```
syms alpha_2o alpha_6o tau_7o
eqns_SS = subs( ode_red, {alpha2(t),alpha6(t),tau7(t)},
{alpha_2o,alpha_6o,tau_7o})
```

```
eqns_SS =
```

$$\begin{pmatrix} 0 = V_0 - \alpha_{2o} + \frac{\tau_{7o}}{3} \\ 0 = \frac{\tau_{7o}}{200} - \frac{981}{100} \\ 0 = -\frac{400000 \alpha_{2o}}{3} \end{pmatrix}$$

Compute the initial conditions: that is the value of V0 that keeps the mass fixed.

```
sol_ics = solve(eqns_SS,{alpha_2o,V0,tau_7o})
```

```
sol_ics = struct with fields:
    alpha_2o: 0
           V0: -654
    tau_7o: 1962
```

```
% Add to the voltage a step
%eqns_with_all_data = subs(tmp,{V0},{sol_ics.V0-1})
% tranfrom in Matlab function for numerical solution
[MM,FF] = massMatrixForm(ode_red,ode_vars);
ode_rhs = odeFunction(FF,ode_vars,V0)
```

```
ode_rhs = function_handle with value:
    @(t,in2,param1)[-in2(1,:)+param1+in2(3,:)/3.0;in2(3,:)/2.0e+2-9.81e+2./1.0e+2;in2(1,:).*(-1.3333333333333333)]
```

```
% Initial velocity v1(t) different equal to 0.1
ics = [0.0; % voltage capacitor(0)
       0.0; % mass speed(0)
       double(sol_ics.tau_7o)]; % spring force value (0)
tspan = [0, 10]; % Define the time span for the simulation
```

```
function v_cmd = voltage_command(t,V0)
    if t<2
        v_cmd = V0;
    else
```



```

if t < 6
    v_cmd = V0-10;
else
    v_cmd = V0;
end
end
end
voltage_command(0:0.1:10,double(sol_ics.V0))

```

```

ans =
-654

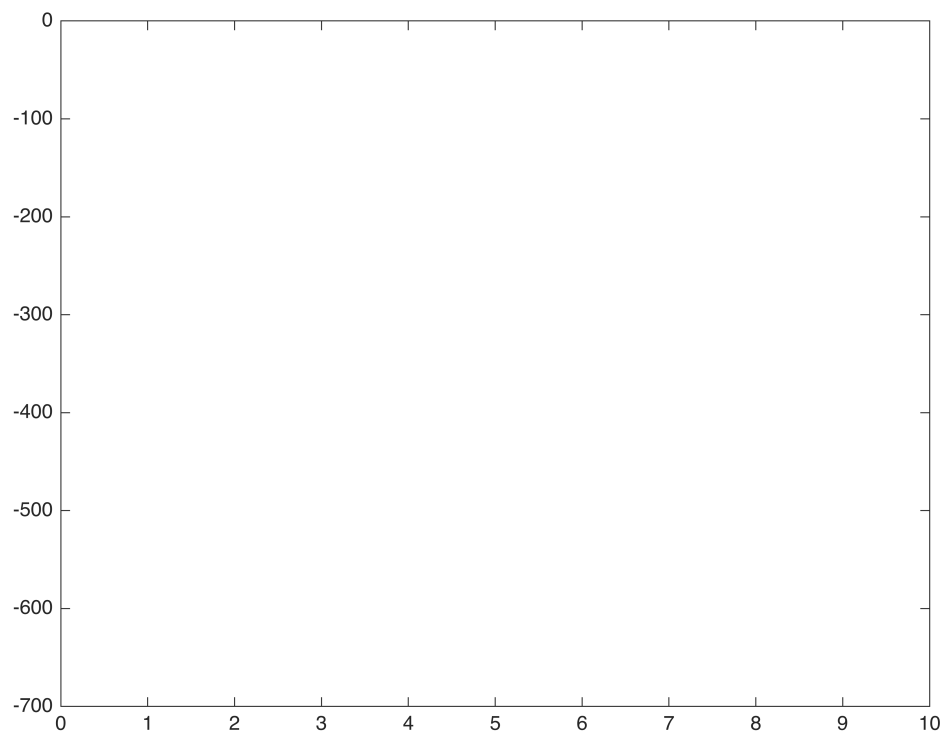
```

Function of command on Voltage

```

figure
plot(0:0.1:10,voltage_command(0:0.1:10,double(sol_ics.V0)), 'Linewidth',2)
ylim([-700 0])

```



```

% call Runge Kutta integrator
[tSol, ySol] =
ode45(@(t,x)ode_rhs(t,x,voltage_command(t,double(sol_ics.V0))), tspan,
ics); % Solve the ODE using ode45

% Plot the results
figure;
tiledlayout(3,1)

```

```

nexttile
plot(tSol, ySol(:,1),'LineWidth',2);
xlabel('Time (s)');
ylabel('[V]');
title('ODE Solution capacitor voltage');
nexttile
plot(tSol, ySol(:,2),'LineWidth',2);
xlabel('Time (s)');
ylabel('[m/s]');
title('ODE Solution of mass speed');
nexttile
plot(tSol, ySol(:,3),'LineWidth',2);
xlabel('Time (s)');
ylabel('[N]');
title('ODE Solution spring force');

grid on;

```

